

Data Structures in Java

Lecture 5: Iterators. Linked Lists. Inner classes.

9/21/2016

Daniel Bauer

Review: Java Generics

- We don't normally know what kind of object to expect in a data structure.
- Java allows to add *type parameters* (<> *syntax*) to definitions of classes. Such classes are called *generic classes*.

```
public class MyArrayList<AnyType> {  
    private AnyType[] theItems;  
    ...  
    public AnyType get(int idx) { ... }  
    public boolean add(int idx, AnyType x) { ... }  
}
```

Review Java Generics (2)

- Type parameters make it possible to create a new data structure for specific objects (and their subtypes) during runtime.

```
List<Integer> l = new MyArrayList<Integer>();
```

- In Java 7 and 8, this can be simplified using the <> (Diamond) operator:

```
List<Integer> l = new MyArrayList<>();
```

- Type of l is inferred automatically.

The Java Collection API

```
package java.util;

interface Collection<E> extends Iterable<E> {
    boolean add(E e);
    boolean addAll(Collection<? extends E> c);
    void clear();
    boolean contains(Object o);
    boolean containsAll(Collection<?> c);
    boolean isEmpty();
    Iterator<E> iterator(); // via Iterable
    boolean remove(Object o);
    boolean removeAll(Collection<?> c);
    boolean retainAll(Collection<?> c);
    int size();
    Object[] toArray();
    <T> T[] toArray(T[] a);
}
```

For-Each Loops

- Iterables support special Java syntax

```
for (T item : someIterable) {  
    System.out.println(item.toString());  
}
```

- No need to explicitly get the `Iterator` and call `next()` repeatedly.

Java Iterators

```
package java.lang;

interface Iterator<T> {
    boolean hasNext();
    T next();
    void remove();
}
```

Our List implementation should be compatible with the Iterator interface.

The Iterable Interface

```
package java.lang;

interface Iterable<T> {
    Iterator<T> iterator();
}
```

- Using Iterables and Iterators:

```
Iterator<T> someIterator = someIterable.iterator()

while (someIterator.hasNext()) {
    T nextItem = someIterator.next();
    System.out.println(nextItem.toString());
}
```

- Don't implement Iterable and Iterator in the same class!

Nested Classes

- Usually every Java class is defined in its own .java file.
- Sometimes classes have a specific purpose (in relation to another class) and are not used anywhere else.

```
class OuterClass {  
    ...  
    static class StaticNestedClass {  
        ...  
    }  
    class InnerClass {  
        ...  
    }  
}
```


Inner Classes

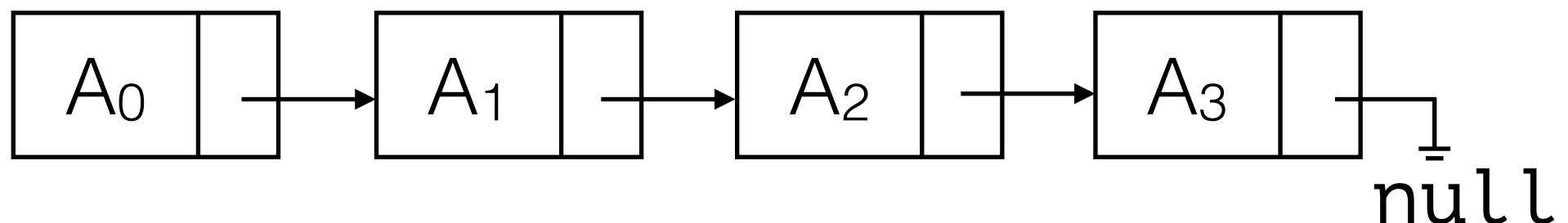
```
public class MyArrayList<AnyType> implements Iterable<AnyType>{  
  
    . . .  
  
    public java.util.Iterator<AnyType> iterator( ) {  
        return new ArrayListIterator( );  
    }  
  
    private class ArrayListIterator implements java.util.Iterator<AnyType> {  
        private Node<AnyType> current = beginMarker.next;  
        . . .  
    }  
}
```

- Instances of inner classes can access instance members of the outer instance that created it.

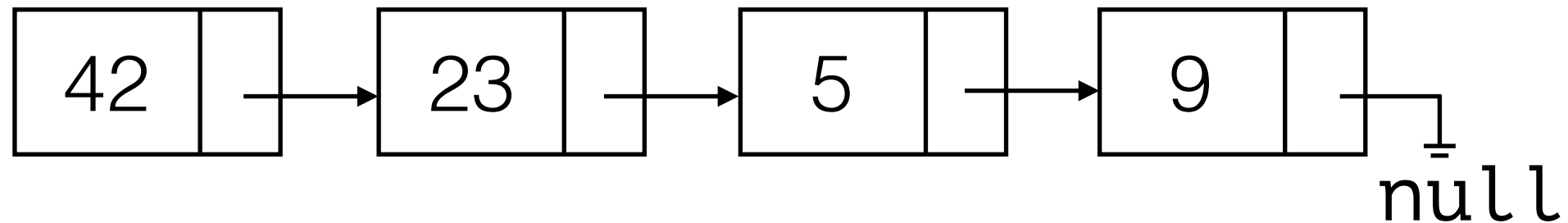
Simple Linked Lists

- Series of *Nodes*. Each *Node* contains:
 - A reference to the data object it contains.
 - A reference to the next node in the List.

```
public class Node {  
    public Integer data;  
    public Node next;  
    public Node(Integer d, Node n) {  
        data = d;  
        next = n;  
    }  
}
```



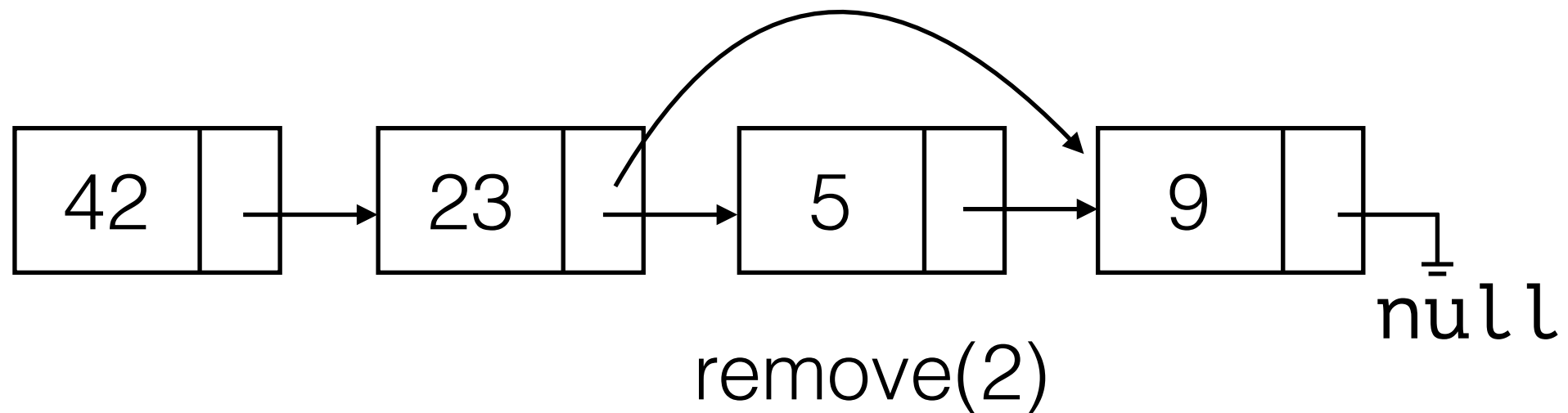
Running Time for Simple Linked List Operations



printList	N
find(x)	N
findKth(k)	k
next()	1

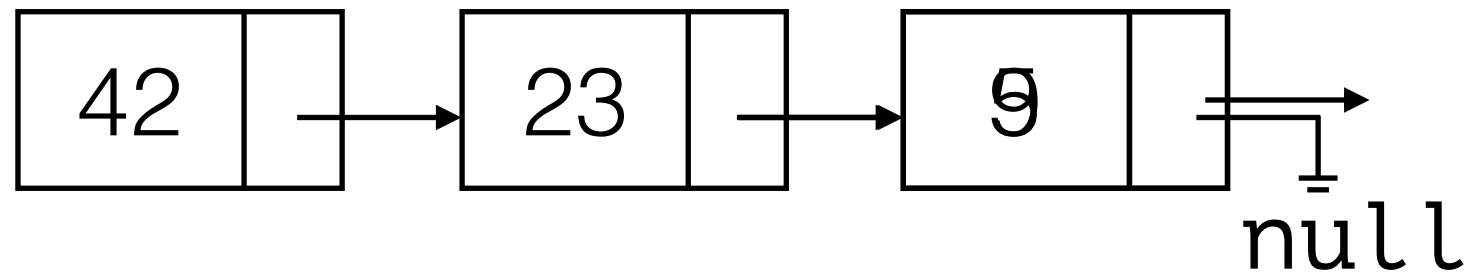
In many applications we can use next() instead of findKth(k).
(for every element in the list do... / filter the list ...)

Simple Linked List Removal



findKth(k)	k
next()	1
insert(x,k)	
remove(k)	search time + 1

Simple Linked List Insertion



insert(5,2)

insert(x,k)	search time +1
-------------	----------------

Inserting in position 0?

Inserting in position N-1?

Linked list should remember the first and last object.

Static Nested Classes

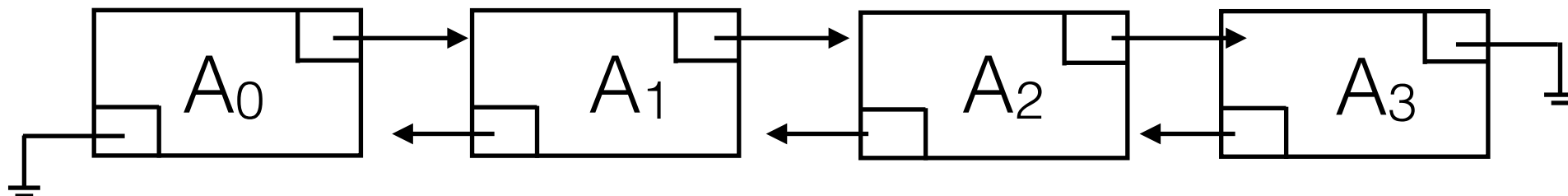
```
public class MyLinkedList<AnyType> implements Iterable<AnyType>{  
    private static class Node<AnyType> {  
        public Node( AnyType d, Node<AnyType> p, Node<AnyType> n ) {  
            data = d; prev = p; next = n;  
        }  
        public AnyType data;  
        public Node<AnyType> prev;  
        public Node<AnyType> next;  
    }  
    . . .  
}
```

- Static nested classes cannot access any instance members of the outer class.
- They essentially behave like normal top-level classes.

Doubly Linked Lists

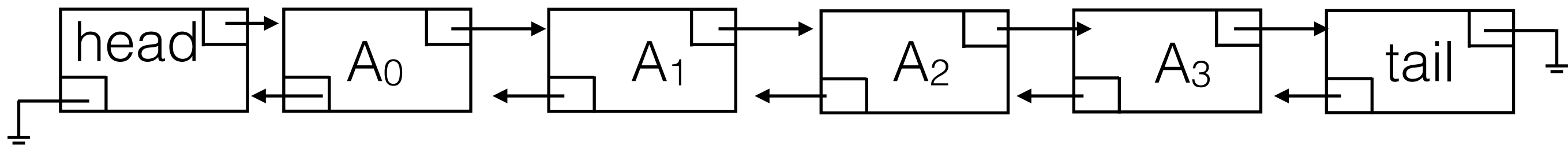
- Also maintain reference to previous node in the list.
- Speeds up append at end of list.

```
public class Node {  
    public Integer data;  
    public Node next;  
    public Node prev;  
    public Node(Integer d, Node n, Node p) {  
        data = d; next = n; prev = n;  
    }  
}
```

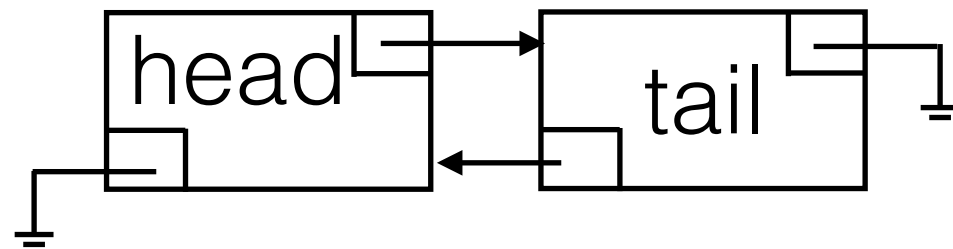


Doubly Linked List with Sentinel Nodes

- header node, tail node
- make implementation of `next` / `previous` easier.
- Remove other special cases
(e.g. removing first node/last node)



Empty Doubly Linked List with Sentinel Nodes



Lists in the Java API

